

Unidad 2: Patrones de diseño

Semana 2 Unidad 2: Patrones de diseño (con criterio y práctica)

1. Patrón vs anti patrón. Contexto, fuerzas, trade offs. Cuándo NO usar patrones.
2. Patrones creacionales: Factory Method, Abstract Factory, Builder, Singleton (uso responsable).
3. Patrones estructurales: Adapter, Facade, Proxy, Composite.
4. Patrones de comportamiento: Strategy, Observer, Command, State, Template Method, Iterator.
5. Patrones de arquitectura ligeros: capas, Clean/Hexagonal (introducción aplicada al proyecto).
6. ADR (Architecture Decision Records): documentación breve de decisiones técnicas.
7. Relación entre SOA y los Servicios Web. Tecnologías que apoyan decisiones arquitectónicas.

Bibliografía obligatoria:

Bases de datos avanzadas e ingeniería del software de Jorge López Querol, Eva María Campos Monge y Maribel Campos Monge.

Bibliografía complementaria:

Shaw, M., & Garlan, D. (1996). Software architecture: perspectives on an emerging discipline (Vol. 1, p. 12). Englewood Cliffs: Prentice Hall

Bass, L., Clements, P., Kasman, R., Bass, K. (2012). Software Architecture in Practice (SEI Series in Software Engineering). 3ra. ed. Addison-Wesley Pub Co.

Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., ... & Stafford, J. (2010). Documenting Software Architectures: Views and Beyond, Portable Documents. Pearson Education.

Clements, P. (2002) Evaluating Software Architectures: Methods and Case Studies. et al. 1ra. ed. Addison-Wesley.

Buschmann, F., Henney, K., & Schmidt, D. (2007). Pattern-oriented Software Architecture: On Patterns and Pattern Language (Vol. 5). John Wiley & sons

Schmidt, D. C., Stal, M., Rohnert, H., & Buschmann, F. (2013). Pattern-Oriented Software Architecture, Patterns for Concurrent and Networked Objects(Vol. 2). John Wiley & Sons.

2- Patrones Creacionales

Resuelven el problema de cómo se crean los objetos sin acoplar el sistema a implementaciones concretas.

Factory Method

Problema: No quiero acoplarme a clases concretas.

Solución: Delegar la creación a subclases.

Fuerza: Extensibilidad vs simplicidad

Trade-off:

Abierto a extensión

– Más clases

Factory Method

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/0f8dae04-c80d-4cb6-b3b5-85a07e3988e9>

Abstract Factory

Problema: Crear familias de objetos relacionados.

Ejemplo: UI Windows vs Mac.

Fuerza: Consistencia vs flexibilidad individual.

Trade-off:

Cohesión entre productos

– Difícil agregar nuevos tipos

Abstract Factory

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/a8060171-9bd3-4818-8bbb-3e7150d140cf>

Builder

Problema: Construcción compleja paso a paso.

Ej: armar objeto con 10 parámetros opcionales.

Fuerza: Claridad vs verbosidad.

Builder

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/8671fd8a-047e-43ff-b0c0-1809ac3e7007>

Singleton (uso responsable)

Problema: Garantizar una única instancia.

Ej: logger, config.

Fuerza: Control global vs testabilidad.

Problema arquitectónico

- Acoplamiento oculto
- Dificulta testing
- Puede convertirse en estado global

En arquitectura moderna se prefiere inyección de dependencias.

Singleton

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/0947ae6f-650b-4105-a411-e04fe5b6cc64>

3- Patrones Estructurales

Organizan cómo se relacionan los componentes.

Adapter

Problema: Interfaces incompatibles.

Solución: Adaptar una interfaz a otra.

Fuerza: reutilización vs pureza de diseño.

Adapter

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/e26a617c-9bb4-4f05-8dd6-15db2c592203>

Facade

Problema: Sistema complejo.

Solución: Interfaz simplificada.

Fuerza: simplicidad vs control fino.

Muy usado en arquitectura por capas.

Facade

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/9bb599c5-5ee7-430b-9134-4c92d37037c7>

Decorator

Problema: Agregar comportamiento dinámicamente.

Fuerza: Extensión sin herencia vs complejidad de composición.

Proxy

Problema: Controlar acceso a un objeto.

Tipos:

- Lazy loading
- Seguridad
- Remoto

Fuerza:

Control vs transparencia.

Proxy

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/34aef31e-f702-4ce3-887b-238eb0eac4a2>

Composite

Problema: Tratar objetos individuales y compuestos igual.

Ej: Árbol de archivos, Estructuras jerárquicas

Fuerza: Uniformidad vs complejidad estructural.

Composite

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/317fa593-8627-4593-83df-28e597369adb>

4- Patrones de Comportamiento

Definen cómo interactúan los objetos.

Strategy

Problema: Variar algoritmo en tiempo de ejecución.

Fuerza: Flexibilidad vs cantidad de clases.

Ej Método de pago.

Strategia

Enlace Notebooklm: <https://notebooklm.google.com/notebook/72378dcd-b6a7-480e-ae12-b0d637d605e5>

Observer

Problema: Dependencias uno-a-muchos.

Ej: Eventos, Reactividad

Fuerza: Desacoplamiento vs riesgo de cascadas.

Base conceptual para: Event-driven architecture

Observer

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/53c9e549-5304-4347-b72e-cedf1375075b>

Command

Problema: Encapsular acciones como objetos.

Permite:

- Undo
- Logging
- Colas

Fuerza: Flexibilidad vs overhead.

Command

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/47dd12db-8bb4-4a4e-b16c-59bd340b2578>

State

Problema: Comportamiento dependiente del estado.

Evita: if/else gigantes.

Fuerza: Claridad vs más clases.

State

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/038fae5d-8568-4928-a8e5-3c9af207eb7b>

Template Method

Problema: Algoritmo fijo con pasos variables.

Fuerza: Reutilización vs rigidez estructural.

Template

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/bf0cea1e-ee1e-4837-88eb-17932e59b144>

Iterator

Problema: Recorrer colecciones sin exponer estructura interna.

Fuerza: Encapsulamiento vs capa extra.

Iterator

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/2f60702c-19cc-4fa3-91c8-6e0b5c030b56>

Comparación por Tipo

Tipo	Qué Resuelven	Fuerza Principal	Riesgo
Creacionales	Cómo se crean objetos	Flexibilidad vs simplicidad	Exceso de abstracción
Estructurales	Cómo se relacionan	Reutilización vs complejidad	Diseño innecesariamente complejo
Comportamiento	Cómo interactúan	Desacoplamiento vs trazabilidad	Difícil seguimiento del flujo

5-Patrones de Arquitectura “ligeros”

Aquí entramos más en el espíritu de Shaw & Garlan: estilos arquitectónicos.

Arquitectura en Capas

Ej:

- Presentación
- Negocio
- Persistencia

Fuerzas: Modularidad vs rendimiento.

Ventajas:

- Separación de responsabilidades
- Testeo

Desventaja:

- Puede generar rigidez

Clean Architecture

Inspirada en separación de dependencias.

Regla central:

Las dependencias apuntan hacia el dominio.

Capas típicas:

- Entidades
- Casos de uso
- Interfaces
- Frameworks

Fuerza: Independencia del framework vs complejidad conceptual.

Arquitectura Hexagonal (Ports & Adapters)

Sistema central aislado.

Conceptos:

- Núcleo
- Puertos
- Adaptadores

Fuerza: Testabilidad vs mayor diseño inicial.

6-ADR (Architecture Decision Records)

Concepto moderno alineado al pensamiento de Shaw & Garlan:

Documentar decisiones arquitectónicas clave.

Formato típico:

- # Título
- ## Contexto
- ## Decisión
- ## Consecuencias

Ejemplo:

Contexto: Necesitamos desacoplar base de datos.

Decisión: Usar Repository Pattern.

Consecuencia: Mayor abstracción, menor dependencia directa.

Comparación Específica

Patrón	Problema que Resuelve	Fuerza en Juego	Trade-Off	Cuándo NO usar
Factory Method	Evitar acoplar a clases concretas	Extensión vs simplicidad	Más clases	Si no habrá variantes
Abstract Factory	Familias de objetos relacionados	Consistencia vs flexibilidad	Difícil agregar productos nuevos	Si solo hay un tipo
Builder	Construcción compleja	Claridad vs verbosidad	Código más largo	Objetos simples
Singleton	Una única instancia	Control global vs testabilidad	Acoplamiento oculto	Sistemas concurrentes complejos
Adapter	Interfaces incompatibles	Reutilización vs pureza	Capa extra	Si puedes modificar la clase original
Facade	Simplificar subsistemas	Usabilidad vs control	Puede ocultar funcionalidad	Si el sistema ya es simple
Decorator	Agregar comportamiento dinámico	Flexibilidad vs claridad	Difícil de depurar	Si el comportamiento es fijo
Proxy	Controlar acceso	Seguridad vs transparencia	Más complejidad	Si no hay necesidad de control
Composite	Jerarquías árbol	Uniformidad vs claridad	Diseño más abstracto	Estructuras planas
Strategy	Variar algoritmos	Flexibilidad vs cantidad de clases	Proliferación de estrategias	Si el algoritmo no cambia
Observer	Notificaciones automáticas	Desacople vs control del flujo	Cascadas de eventos	Si el flujo debe ser explícito

Command	Encapsular acciones	Flexibilidad vs overhead	Más objetos	Si no se requiere historial/undo
State	Comportamiento por estado	Claridad vs complejidad	Más clases	Si los estados son pocos
Template Method	Algoritmo base con variaciones	Reutilización vs rigidez	Dependencia jerárquica	Si el algoritmo cambia mucho

Abstract Factory

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/a8060171-9bd3-4818-8bbb-3e7150d140cf>

Adapter

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/e26a617c-9bb4-4f05-8dd6-15db2c592203>

Builder

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/8671fd8a-047e-43ff-b0c0-1809ac3e7007>

Cadena de Responsabilidad

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/cd1026b5-83ab-40a9-8a7c-78eb9cdfaa6b>

Command

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/47dd12db-8bb4-4a4e-b16c-59bd340b2578>

Composite

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/317fa593-8627-4593-83df-28e597369adb>

Facade

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/9bb599c5-5ee7-430b-9134-4c92d37037c7>

Factory Method

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/0f8dae04-c80d-4cb6-b3b5-85a07e3988e9>

Iterator

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/2f60702c-19cc-4fa3-91c8-6e0b5c030b56>

Memento

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/e7f29766-5dc4-4a5a-bf24-b3b66963700b>

Observer

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/53c9e549-5304-4347-b72e-cedf1375075b>

Proxy

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/34aef31e-f702-4ce3-887b-238eb0eac4a2>

Singleton

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/0947ae6f-650b-4105-a411-e04fe5b6cc64>

State

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/038fae5d-8568-4928-a8e5-3c9af207eb7b>

Strategia

Enlace Notebooklm: <https://notebooklm.google.com/notebook/72378dcd-b6a7-480e-ae12-b0d637d605e5>

Template

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/bf0cea1e-ee1e-4837-88eb-17932e59b144>

Visitor

Enlace a Notebooklm: <https://notebooklm.google.com/notebook/71335f0c-58db-4228-8c07-5cd49ad8e606>